

Class 08 — Student Notes

Bong Study Hub — Foundation Batch 2026

Making Decisions: if / elif / else

The First Code That "Thinks" Before Acting

These notes summarize what we covered in class. Use them to revise, not to learn the topic for the first time — the real learning happened at the keyboard, watching the same program behave two different ways. No nested if statements, no loops, and no `input()` appear here — those come in later classes.

The Big Question

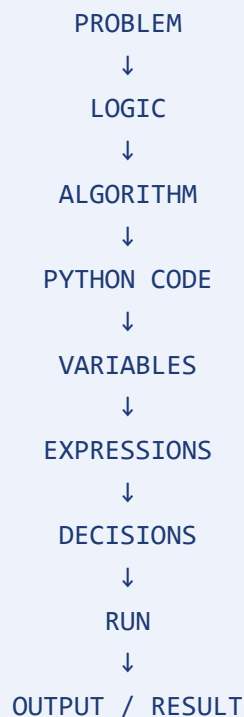
Class 07 ended with a question we didn't answer: every comparison and logical result just gets printed — it never changes what the program does next.

"How do we make a program actually behave differently depending on what's true?"

Quick connection to Class 01

Class 01's ALGORITHM box was always drawn as a straight line — one step after another. Today widens that picture:

ALGORITHM (Class 01 — a straight line) → **algorithms can branch** (today)



1. What Is a Conditional?

A **conditional** is a statement that runs a block of code only if a **condition** — a boolean expression — is `True`. If the condition is `False`, that block is skipped entirely.

2. Your First if Statement

```
is_raining = True

if is_raining:
    print("Bring an umbrella!")
```

`if is_raining:` — a condition, then a colon. The line underneath is **indented** — that tells Python it belongs *inside* the `if`.

Run it with `is_raining = True` → the message prints. Change it to `is_raining = False` and run again → **nothing prints**.

Same code. Different data. Completely different behavior — the first time that's happened in this course.

3. Indentation: Not Just Style

Python uses **indentation** — not curly braces `{ }`, not the word `end` — to mark which lines belong to a block. This is real syntax, not a formatting choice.

```
if is_raining:
    print("Bring an umbrella!")
```

→ `IndentationError: expected an indented block`

Every line in the same block needs the **same** indentation:

```
if is_raining:
    print("Bring an umbrella!")
    print("Don't forget your boots!")
```

→ also an `IndentationError` — the second line's indentation doesn't match the first.

Most editors indent four spaces automatically after a colon — let your editor help you, and stay consistent.

4. else: The Otherwise

```
is_raining = False

if is_raining:
    print("Bring an umbrella!")
else:
    print("Enjoy the sunshine!")
```

`else:` means "otherwise" — its block runs only when the `if` condition was `False`. **It never takes its own condition** — it's simply whatever's left.

Exactly one of the two blocks runs, every time — never both, never neither.

5. `elif`: More Than Two Paths

```
age = 15

if age < 13:
    print("Child ticket")
elif age < 20:
    print("Teen ticket")
elif age < 65:
    print("Adult ticket")
else:
    print("Senior ticket")
```

`elif` is short for "**else if**" — check another condition, but only if every condition above it was `False`. With `age = 15`: is it under 13? No. Is it under 20? Yes — `Teen ticket` prints, and Python never even looks at the remaining conditions.

6. Order Matters in `elif` Chains

```
score = 95

if score >= 60:
    print("Pass")
elif score >= 90:
    print("Pass with Distinction")
```

`score` is 95 — that's both `>= 60` and `>= 90`. This prints `Pass`, not `Pass with Distinction` — Python checked the *first* condition, found it `True`, ran that block, and never even looked at the `elif` below it.

Order matters. The more specific condition needed to come first.

7. Comparisons and Logical Operators Inside Conditions

```
age = 16
is_student = True

if age >= 13 and age <= 19 and is_student:
    print("Eligible for the student teen discount!")
```

This condition is built entirely from Class 07's toolkit — two comparisons and a logical `and`, combined into one boolean expression. An `if` just needs a boolean expression, however it's built.

8. Common Structure Patterns

```
if alone → do something, or nothing
if / else → exactly one of two paths
if / elif / else → exactly one of several paths
```

Choosing between them is about how many distinct outcomes your problem actually has.

9. Connecting Back to Class 01

Class 01's `ALGORITHM` was always a straight line — one step after another. Today, that picture gets wider: an algorithm can **branch** — different data can send it down a different path entirely.

Try It Yourself — Your Profile Card Decides

Open your Profile Card from Classes 06–07. Add:

- An **if/else** using your `is_student` boolean — one message if `True`, a different message if `False`.
- An **if/elif/else** chain using your `age` variable, with at least three branches — e.g.:

```
        if age < 13:
            print("Category: Child")
        elif age < 20:
            print("Category: Teen")
        elif age < 65:
            print("Category: Adult")
        else:
            print("Category: Senior")
```

Run it, then **change one variable's value and run it again** — confirm a different branch actually executes.

Quick Recap

- A **conditional** runs a block only if a **condition** is `True`.
- An **if** block's indented lines run only when its condition is `True`.
- **Indentation is syntax** in Python, not decoration — inconsistent indentation causes an `IndentationError`.
- **else** runs only when every condition above it was `False` — it never takes its own condition.
- **elif** ("else if") checks another condition, only if the ones above it were `False`.
- In an **elif** chain, **only the first true branch runs** — order matters.
- A condition can be built from Class 07's comparison and logical operators directly.
- Today's branching idea widens Class 01's **ALGORITHM** — it's not just a straight line of steps anymore.

Key Terms

Term	Meaning
Conditional	A statement that runs a block of code only if a condition is true.
Condition	A boolean expression that controls whether a block runs.
Block	A group of statements, indented consistently, that belong together.
Indentation	The whitespace before a line, which in Python defines which block it belongs to.
<code>if</code>	Runs its block only when its condition is <code>True</code> .
<code>else</code>	Runs its block only when every condition above it was <code>False</code> .

<code>elif</code>	Short for "else if" — checks another condition, only if the ones above it were <code>False</code> .
Branch / Branching	A program following one of several possible paths, depending on data.

Think About It

Try answering these in your own words — you don't need exact wording, just the right idea.

1. When does an `if` block's indented line actually run?
2. Why does indentation matter so much in Python?
3. When does an `else` block run?
4. If more than one `elif` condition is true, which one actually runs?
5. Can a condition use `and/or/not` from Class 07?
6. How does today's idea connect back to Class 01's `ALGORITHM`?

Final Takeaway

"Every program before today ran the same way, every single time. Today, for the first time, your code actually thinks before it acts — it looks at its own data and chooses a path. That single idea — branching — is the foundation of almost everything a real program does."

Right now, every decision only runs once. What if a program needed to make the same decision — or repeat the same action — over and over, without you writing it out by hand each time? That's exactly where we pick up next class.